코드 설명 보고서

PyTorch를 활용한 MNIST 손글씨 숫자 분류기 구현 및 학습 성능 분석

학번 : 2026406076

이름 : 정창규

제출일 : 2026-10-29

목차

1. 과제 개요 및 목적

2. MNIST 분류 모델 구현

- 2.1 데이터셋 준비 및 모델 구성
- 2.2 학습 및 평가 과정
- 2.3 학습 결과 시각화

3. 실험 결과 및 분석

- 3.1 학습 조건별 성능 비교
- 3.2 오분류 사례 분석

4. 결론 및 고찰

5. AI 활용 내역 및 참고 자료

1. 과제 개요 및 목적

이번 과제에서는 PyTorch를 사용하여 MNIST 손글씨 숫자를 분류하는 신경망 모델을 구현했다. MNIST는 0부터 9까지의 손글씨 숫자로 구성된 데이터셋으로, 각 이미지는 28×28 크기의 흑백 이미지이다. 이 데이터를 이용하여 입력된 이미지가 어떤 숫자인지 예측하는 MLP 신경망을 만들었다.

모델은 입력층 784개, 은닉층 128개, 출력층 10개의 구조로 구성했다. 학습 과정에서는 순전파를 통해 예측값을 계산하고, 손실 함수를 이용하여 실제 정답과의 차이를 구했다. 이후 역전파로 기울기를 계산하고 Adam Optimizer를 사용하여 가중치와 편향을 갱신하도록 했다.

기본 모델을 구현한 후에는 Epoch와 Learning Rate를 변경하여 총 3가지 조건으로 학습을 진행했다. 각 조건에서 나온 Loss와 Accuracy를 비교하고, 학습 과정에서 Loss가 어떻게 변하는지 그래프로 확인했다. 또한 모델이 잘못 예측한 손글씨 숫자 9개를 추출하여 실제 정답과 예측 결과를 비교했다.

2. MNIST 분류 모델 구현

2.1 데이터셋 준비 및 모델 구성

신경망 학습에 필요한 데이터를 준비하기 위해 MNIST 데이터셋을 불러왔다. 이후 전체 학습 데이터를 학습용과 검증용으로 나누었다.

-코드 7~16줄

```
# 실험 재현을 위한 랜덤 시드 설정
torch.manual_seed(0)

# MNIST 학습 데이터 불러오기 및 텐서 변환
full = datasets.MNIST("data", train=True, download=True,
                transform=transforms.ToTensor())

# 학습 데이터 55,000개와 검증 데이터 5,000개로 분리
train, val = random_split(full, [55000, 5000],
    generator=torch.Generator().manual_seed(0))
```

MNIST 데이터셋은 datasets.MNIST()를 이용하여 불러왔으며, transforms.ToTensor()를 통해 이미지 데이터를 텐서로 변환했다. 전체 학습 데이터 60,000개 중 55,000개는 모델 학습에 사용하고, 나머지 5,000개는 검증 데이터로 분리했다. 또한 랜덤 시드를 0으로 설정하여 실행할 때마다 데이터가 동일하게 분리되도록 했다.

이어서 학습이 끝난 모델을 평가하기 위한 테스트 데이터를 별도로 불러왔다.

-코드 18~20줄

```
# MNIST 테스트 데이터 불러오기
test = datasets.MNIST("data", train=False, download=True,
                transform=transforms.ToTensor())
```

테스트 데이터는 train=False로 설정하여 불러왔다. 총 10,000개의 이미지로 구성되어 있으며, 학습 과정에는 사용하지 않고 모델의 최종 분류 정확도를 확인할 때 사용했다.

마지막으로 데이터를 일정한 개수씩 처리하기 위해 DataLoader를 구성했다.

-코드 22~29줄

```
# 학습 데이터를 64개씩 묶고 순서를 무작위로 섞음
train_loader = DataLoader(train, batch_size=64, shuffle=True)

# 검증 데이터를 256개씩 묶음
val_loader = DataLoader(val, batch_size=256)

# 테스트 데이터를 256개씩 묶음
test_loader = DataLoader(test, batch_size=256)
```

학습 데이터는 한 번에 64개씩 처리하고, 검증 데이터와 테스트 데이터는 256개씩 처리하도록 했다. 학습 데이터에는 shuffle=True를 적용하여 매 Epoch마다 데이터 순서가 섞이도록 했으며, 검증과 테스트 데이터에는 적용하지 않았다.

데이터 준비를 마친 뒤에는 모델 학습에 사용할 장치를 설정했다.

-코드 32~34줄

```
# CPU 또는 GPU 학습 장치 설정
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu")
```

torch.cuda.is_available()을 이용하여 GPU 사용 가능 여부를 확인했다. GPU를 사용할 수 있으면 CUDA를 선택하고, 그렇지 않으면 CPU에서 학습하도록 설정했다.

다음으로 MNIST 손글씨 숫자를 분류할 MLP 신경망을 구성했다.

-코드 36~42줄

```
# MLP 신경망 모델 구성 (784 → 128 → 10)
model = nn.Sequential(
    nn.Flatten(),          # 28×28 이미지를 784개 값으로 변환
    nn.Linear(784, 128), # 입력값 784개를 은닉층 128개로 변환
    nn.ReLU(),             # 활성화 함수 적용
    nn.Linear(128, 10)    # 숫자 0~9에 대한 점수 출력
).to(device)
```

신경망은 784 → 128 → 10 구조의 MLP 모델로 만들었다. 먼저 nn.Flatten()을 사용하여 28×28 크기의 이미지를 784개의 값으로 펼쳤다. 이후 nn.Linear(784, 128)을 통해 128개의 뉴런으로 구성된 은닉층을 거치고, nn.ReLU() 활성화 함수를 적용했다. 마지막으로 nn.Linear(128, 10)에서 숫자 0부터 9까지의 점수 10개를 출력하도록 했다.

2.2 학습 및 평가 과정

MLP 신경망을 구성한 다음에는 모델 학습에 필요한 손실 함수와 Optimizer를 설정했다.

-코드 44~51줄

```
# 손실 함수 설정
criterion = nn.CrossEntropyLoss()

# Adam Optimizer 및 학습률 설정
optimizer = torch.optim.Adam(model.parameters(), lr=0.0005)

# 학습 반복 횟수 설정
epochs = 5
```

손실 함수는 nn.CrossEntropyLoss()를 사용했으며, Optimizer는 Adam을 선택했다. 코드에 표시된 학습률 0.0005와 Epoch 5는 실험 3에서 사용한 설정이다. 실험 1과 2에서는 학습률을 0.001로 설정하고 Epoch를 각각 5와 10으로 변경하여 결과를 비교했다.

이어서 검증 데이터와 테스트 데이터의 손실값과 정확도를 계산하는 함수를 작성했다.

-코드 54~74줄

```
# 검증 및 테스트 데이터 평가 함수
def evaluate(loader):
    model.eval()  # 모델을 평가 모드로 전환
    loss_sum, correct, count = 0.0, 0, 0

    # 평가 중에는 기울기를 계산하지 않음
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            logits = model(images)  # 예측 점수 계산

            # 전체 손실 합산
            loss_sum += criterion(logits, labels).item() * len(labels)

            # 예측값과 정답을 비교하여 맞힌 개수 계산
            correct += (logits.argmax(1) == labels).sum().item()
            count += len(labels)

    # 평균 Loss와 Accuracy 반환
    return loss_sum / count, correct / count
```

검증과 테스트에 사용할 evaluate() 함수를 작성했다. model.eval()로 모델을 평가 모드로 전환하고, torch.no_grad()를 사용하여 평가 과정에서는 기울기를 계산하지 않도록 했다. 모델의 예측값과 실제 정답을 비교하여 정확도를 구하고, 전체 데이터에 대한 평균 Loss도 계산하도록 했다.

평가 함수를 작성한 뒤에는 실제 모델을 학습시키는 반복문을 구현했다. 학습 과정에서 Train Loss와 Validation Loss를 기록하기 위해 각각의 리스트를 만들고, 설정한 Epoch 수만큼 학습을 반복하도록 했다.

-코드 77~84줄

```
# Epoch별 학습 및 검증 Loss 저장
train_losses = []
val_losses = []

# 설정한 Epoch 수만큼 전체 학습 반복
for epoch in range(epochs):
    model.train()  # 모델을 학습 모드로 설정
    total_loss = 0.0  # Epoch별 손실 합계 초기화
```

model.train()으로 모델을 학습 모드로 전환하고, 매 Epoch마다 손실값을 새로

계산할 수 있도록 total_loss를 0으로 초기화했다.

실제 학습은 train_loader에서 데이터를 미니배치 단위로 불러오는 방식으로 진행했다.

-코드 86~98줄

```
# 학습 데이터를 미니배치 단위로 불러오기
for images, labels in train_loader:
    images = images.to(device)  # 이미지 데이터를 학습 장치로 이동
    labels = labels.to(device)  # 정답 데이터를 학습 장치로 이동

    optimizer.zero_grad()  # 이전 기울기 초기화
    logits = model(images)  # Forward: 예측 점수 계산
    loss = criterion(logits, labels)  # Loss 계산
    loss.backward()  # Backpropagation: 기울기 계산
    optimizer.step()  # Optimizer: 가중치와 편향 갱신

    # 미니배치별 손실을 전체 손실에 누적
    total_loss += loss.item() * images.size(0)
```

train_loader에서 학습 데이터를 64개씩 불러와 학습을 진행했다. 먼저 optimizer.zero_grad()로 이전에 계산된 기울기를 초기화하고, model(images)를 통해 숫자별 예측 점수를 계산했다. 이후 손실 함수를 이용하여 Loss를 구하고, loss.backward()로 역전파를 수행했다. 마지막으로 optimizer.step()을 통해 가중치와 편향을 갱신했다. 각 미니배치마다 이 과정을 반복했으며, 각 미니배치의 손실값을 누적하여 Epoch별 평균 Loss를 계산할 수 있도록 했다.

한 Epoch의 학습이 끝날 때마다 평균 Train Loss를 계산하고, 검증 데이터에 대한 Loss와 Accuracy를 확인했다.

-코드 100~113줄

```
# 전체 학습 데이터에 대한 평균 Loss 계산
train_loss = total_loss / len(train)


# 검증 데이터의 Loss와 Accuracy 계산
val_loss, val_acc = evaluate(val_loader)

# Epoch별 Loss 기록
train_losses.append(train_loss)
val_losses.append(val_loss)

# Epoch별 학습 및 검증 결과 출력
print(f"Epoch {epoch + 1}: Train Loss={train_loss:.4f}, "
      f"Val Loss={val_loss:.4f}, Val Accuracy={val_acc:.4f}")
```

한 Epoch의 학습이 끝나면 누적된 손실값을 전체 학습 데이터 개수로 나누어

평균 Train Loss를 계산했다. 이후 evaluate() 함수를 사용하여 검증 데이터의 Loss와 Accuracy를 구했다. 계산된 손실값은 각각 리스트에 저장하고, Epoch마다 Train Loss, Validation Loss, Validation Accuracy를 터미널에 출력하도록 했다.

모든 Epoch의 학습이 끝난 후에는 별도로 준비한 테스트 데이터를 이용하여 최종 성능을 평가했다.

-코드 116~121줄

```
# 학습 완료 후 테스트 데이터로 최종 성능 평가
test_loss, test_acc = evaluate(test_loader)

# 최종 테스트 결과 출력
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc * 100:.2f}%")
```

학습이 끝난 후 evaluate(test_loader)를 사용하여 테스트 데이터 10,000개에 대한 최종 성능을 확인했다. 테스트 데이터는 학습과 검증에 사용하지 않은 데이터로, 모델이 처음 보는 손글씨 숫자도 정확하게 분류하는지 확인하기 위해 사용했다. 계산된 Test Loss와 Test Accuracy는 터미널에 출력했으며, Accuracy는 백분율(%)로 나타냈다.

2.3 학습 결과 시각화

학습 과정에서 Loss가 어떻게 변하는지 확인하기 위해 ChatGPT의 도움을 받아 그래프 출력 코드를 추가했다.

-코드 124~136줄

```
# Epoch별 학습 및 검증 Loss 그래프 생성
plt.plot(range(1, epochs + 1), train_losses, label="Train Loss")
plt.plot(range(1, epochs + 1), val_losses, label="Validation Loss")

# 그래프 제목 및 축 설정
plt.title("MNIST Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

# 그래프 이미지 저장
plt.savefig("loss_graph.png", dpi=300, bbox_inches="tight")
```

Matplotlib을 사용하여 Epoch별 Train Loss와 Validation Loss를 하나의 그래프로

나타냈다. 가로축은 Epoch, 세로축은 Loss로 설정했으며, 학습 데이터와 검증 데이터의 손실값을 비교할 수 있도록 했다. 생성된 그래프는 loss_graph.png 파일로 저장했다.

Loss 그래프로 학습 과정을 확인한 뒤에는 모델이 어떤 숫자를 잘못 예측했는지 살펴보기 위해 오분류 이미지 추출 코드를 추가했다.

-코드 138~165줄

```
# 오분류 이미지 수집
model.eval()
wrong_images = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        # 모델의 예측값 계산
        logits = model(images)
        predictions = logits.argmax(1)

        # 예측이 틀린 이미지 추출
        for i in range(len(labels)):
            if predictions[i] != labels[i]:
                wrong_images.append((
                    images[i].cpu(),
                    labels[i].item(),
                    predictions[i].item()
                ))
```

```
        # 오분류 이미지 9개 수집
        if len(wrong_images) >= 9:
            break

    if len(wrong_images) >= 9:
        break
```

테스트 데이터에서 모델의 예측값과 실제 정답이 다른 이미지를 찾아 따로 저장했다. `argmax(1)`을 이용하여 가장 높은 점수를 가진 숫자를 예측값으로 선택하고, 실제 정답과 일치하지 않는 경우에만 `wrong_images` 리스트에 추가했다. 오분류 이미지가 9개 모이면 반복문이 종료되도록 했다.

추출한 이미지들은 실제 정답과 예측값을 비교하기 쉽도록 한 화면에 배치했다.

-코드 167~179줄

```
# 오분류 이미지 시각화
fig, axes = plt.subplots(3, 3, figsize=(9, 9))

for ax, (image, label, pred) in zip(axes.flat, wrong_images):
    ax.imshow(image.squeeze(), cmap="gray")
    ax.set_title(f"True: {label}, Pred: {pred}")
    ax.axis("off")

plt.tight_layout()

# 오분류 이미지 저장
plt.savefig("misclassified_examples.png", dpi=300)
plt.show()
```

추출한 9개의 이미지는 3행 3열로 배치했으며, 각 이미지 위에 실제 정답(True)과 모델이 예측한 숫자(Pred)를 표시했다. 이를 통해 모델이 어떤 숫자를 혼동했는지 확인할 수 있었으며, 결과는 misclassified_examples.png 파일로 저장했다.

3. 실험 결과 및 분석

3.1 학습 조건별 성능 비교

기본 MLP 모델을 완성한 후 Epoch와 Learning Rate를 변경하면서 총 3번의 실험을 진행했다. 모델 구조와 Batch Size, Optimizer는 동일하게 유지하고 학습 조건에 따라 결과가 어떻게 달라지는지 비교했다.

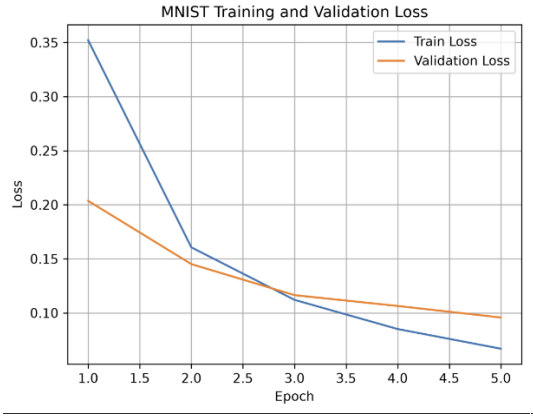
구분	Epoch	Learning Rate	Train Loss	Val Loss	Test Accuracy
Test1	5	0.001	0.0669	0.0959	97.46%
Test2	10	0.001	0.0243	0.0903	97.55%
Test3	5	0.0005	0.1084	0.1211	96.81%

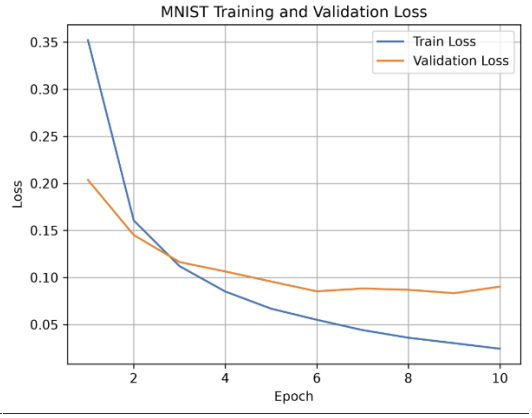
실험 1과 실험 2는 학습률을 0.001로 유지하고 Epoch만 5에서 10으로 늘렸다. 그 결과 Test Accuracy가 97.46%에서 97.55%로 0.09% 증가했다. 다만 실험 2에서는 Epoch 9일 때 Validation Loss가 0.0833으로 가장 낮았고, Epoch 10에서는 0.0903으로 다시 증가했다. 학습을 더 진행하더라도 검증 성능이 계속 좋아지는 것은 아니라는 점을 확인할 수 있었다.

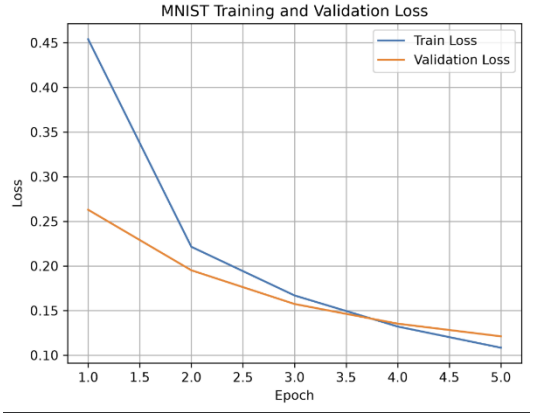
실험 3에서는 Epoch를 5로 유지하고 학습률을 0.0005로 낮췄다. 그 결과 Test Accuracy가 96.81%로 감소했으며, Train Loss와 Validation Loss도 실험 1보다 높게 나타났다. 같은 학습 횟수에서 학습률을 낮추자 학습이 상대적으로 느리게

진행된 것으로 보인다.

이번 실험에서는 Epoch 10, Learning Rate 0.001 조건에서 가장 높은 Test Accuracy인 97.55%를 기록했다.







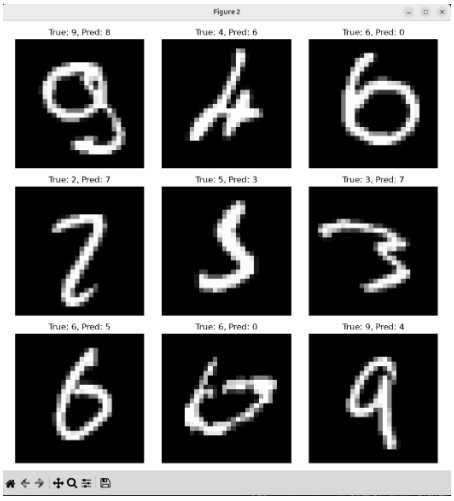
왼쪽 위: 실험 1 (Epoch 5, LR 0.001)

오른쪽 위: 실험 2 (Epoch 10, LR 0.001)

왼쪽 아래: 실험 3 (Epoch 5, LR 0.0005)

3.2 오분류 사례 분석

모델이 어떤 숫자를 잘못 예측했는지 확인하기 위해 테스트 데이터에서 오분류된 이미지 9개를 추출했다. 각 이미지에는 실제 정답(True)과 모델이 예측한 숫자(Pred)를 표시하여 결과를 비교했다.



오분류된 이미지 중에는 숫자 6을 0으로, 숫자 9를 4로, 숫자 5를 3으로 잘못 예측한 사례가 있었다. 이미지들을 살펴보니 숫자의 일부 획이 불분명하거나 다른 숫자와 비슷한 형태로 작성된 경우가 있었다. 모델이 대부분의 숫자를 정확하게 분류했지만, 손글씨의 모양에 따라 일부 숫자는 잘못 예측할 수 있다는 점을 확인했다.

4. 결론 및 고찰

이번 과제에서는 PyTorch를 이용하여 MNIST 손글씨 숫자 분류 모델을 만들고, 학습 조건을 변경하면서 결과를 비교했다. 처음에는 신경망의 학습 과정이 어렵게 느껴졌지만, 직접 코드를 작성하면서 순전파, 손실 계산, 역전파, 가중치 갱신이 어떤 순서로 이루어지는지 이해할 수 있었다.

총 3가지 조건으로 학습을 진행했으며, Epoch 10과 Learning Rate 0.001을 사용한 실험에서 가장 높은 테스트 정확도인 97.55%가 나왔다. 다만 Epoch를 늘려도 정확도가 크게 증가하지는 않았고, 학습 후반부에는 Validation Loss가 다시 높아지는 모습도 나타났다. 학습률을 낮춘 실험에서도 정확도가 감소하는 결과가 나왔다. 이를 통해 학습 횟수와 학습률을 어떻게 설정하느냐에 따라 모델의 성능이 달라질 수 있다는 점을 확인했다.

또한 오분류 이미지를 살펴보면서 정확도가 높더라도 모든 숫자를 올바르게 예측하는 것은 아니라는 점을 알게 되었다. 이번 과제를 통해 단순히 모델을 실행하는 것에서 끝나지 않고, 학습 결과를 비교하고 성능이 달라진 이유를 생각해 볼 수 있었다.

5. AI 활용 내역 및 참고 자료

5.1 AI 활용 내역

MNIST 데이터셋 준비부터 MLP 신경망 구성, 학습 및 평가 과정까지는 교육 자료를 참고하여 직접 코드를 작성했다. 작성한 코드에서 오류가 있는지 확인하거나 이해가 어려운 부분을 설명받을 때 ChatGPT를 활용했다.

학습이 진행되면서 Train Loss와 Validation Loss가 어떻게 변하는지 확인하기 위해 ChatGPT를 이용하여 그래프 생성 코드를 작성했다. 또한 모델이 잘못 분류한 숫자를 확인할 수 있도록 오분류 이미지 9개를 추출하고, 실제 정답(True)과 예측값(Pred)을 표시하는 코드도 ChatGPT의 도움을 받았다.

추가한 코드는 직접 실행하여 정상적으로 작동하는지 확인했고, 출력된 결과를 바탕으로 세 가지 실험의 성능을 비교했다. 보고서의 구성과 코드 설명을 정리하는 과정에서도 ChatGPT를 활용했다.

5.2 참고 자료

1. ROBIT 강화학습 교육 1일차 강의 자료,「딥러닝과 신경망의 학습 원리」
2. PyTorch MNIST 구현 참고 블로그
   https://noru-jumping-in-the-mountains.tistory.com/10